

Тема 1. Същност на „утвърдителното“ програмиране. Декриптивни и императивни програмни езици. Основни принципи на функционалното програмиране. Езици за функционално програмиране.

1. Същност на „утвърдителното“ програмиране

Утвърдителното програмиране (декларативното/ дескриптивно) е подход, при който програмистът описва какъв резултат трябва да се получи, без задаване на стъпките за неговото изчисляване. Фокусът при този стил е върху *описанието на проблема*, а не върху алгоритъма за решаването му. При този подход програмата е набор от правила, изрази или зависимости. Изпълнението се управлява от средата за изпълнение, а не пряко от програмиста. Подходът улеснява разбирането и поддръжката на кода. Декларативното програмиране намалява сложността при големи системи. То позволява по-лесни оптимизации от страна на компилатора или интерпретатора. Често се използва при работа с данни, логически изводи и функционални изчисления. Кодът обикновено е по-кратък и по-изразителен.

Типични примери за декларативен подход са функционалното и логическото програмиране.

Логическият извод е процес, при който от зададени факти и правила се получава ново знание, без то да е описано директно. Системата сама „разсъждава“ и стига до заключение, използвайки логиката, т.е.

- не се описва как да се изпълни алгоритъма
- а само се описва какво е вярно

Пример:

Имаме Факти

1. Всички хора са смъртни.
2. Сократ е човек.

На базата на тези **Факти** правим **Логически** извод

3. Сократ е смъртен.

Това заключение не е записано директно, а се извежда логически.

Реализираме примера чрез Prolog - логически (декларативен) език за програмиране, при който се задават факти и правила, а системата чрез логически извод определя дали дадено твърдение е вярно. В примера се описва, че Сократ е човек и че всички хора са смъртни, след което чрез запитване се проверява дали Сократ е смъртен.

```
human(socrates).  
mortal(X) :- human(X).
```

Запитване:
mortal(socrates)?

Отговор:
true

Системата прави логически извод.

Как работи логическият извод?

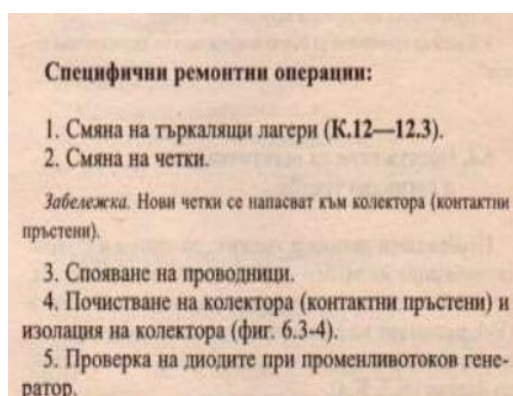
- Има факти – неоспорими твърдения
- Има правила – зависимости между твърдения
- Има запитване (въпрос)
- Системата проверява дали запитването е вярно
- Ако може да го докаже връща резултат

2. Декриптивни и императивни програмни езици

Императивните програмни езици описват програмата като последователност от инструкции. Програмистът указва как точно да се изпълни дадена задача. Използват се променливи, присвояване и промяна на състоянието. Редът на изпълнение на командите е от важно значение.

Примери:

- 1 Първата стъпка от **рецептата за ванилови мъфини** е подготовката на продуктите. Разтопете маслото **на водна баня**, яйцата извадете от хладилника, пресейте брашното, заедно с бакпулвера.
- 2 Разбийте яйцата, заедно със захарта с миксер, тел за разбиване или пък с вилица. Добавете киселото мляко и отново разбъркайте, добавете и маслото. Към течната смес добавете пресятото с бакпулвера брашно малко по малко при непрекъснато разбиване до получаване на еднародна смес. Накрая добавете и есенцията и отново разбийте.
- 3 Във всяко гнездо на **формата за мъфини** слагате по една хартиена кошничка и пълните $\frac{3}{4}$ от обема. Внимавайте да не напълните формите за мъфини прекалено, защото по време на печене мъфините увеличават обема си и ако сместа във формичките е повече от необходимото ще прелее и няма да се изпекат. Ако ползвате силиконова форма за мъфини не е необходимо да слагате хартиени кошнички, само намажете формата с предварително с олио и изсипете сместа.



Дескриптивните езици, от друга страна, описват какво трябва да бъде получено. Липсва явно управление на потока на изпълнение. Състоянието е скрито или строго контролирано.

Пример:



Императивните езици са по-интуитивни за начинаещи. Дескриптивните езици предлагат по-високо ниво на абстракция.

3. Основни принципи на функционалното програмиране

- Основен градивен елемент при Функционалното програмиране са функциите.
- Данните са неизменяеми, което означава, че не се променят след създаване.
- Функциите са чисти и не предизвикват странични ефекти. Резултатът на функцията зависи само от входните ѝ аргументи.
- Функциите са първокласни стойности и могат да се предават като аргументи.
- Използват се функции от висок ред за абстрахиране на повтарящи се операции.
- Рекурсията се използва вместо цикли.

- Принципът на референтна прозрачност улеснява анализа на програмите. Функционалният стил подпомага паралелното и конкурентно изпълнение. Тези принципи водят до по-надежден и лесен за тестване код.

Какво означава една функция да „няма странични ефекти“?

Една функция няма странични ефекти, ако:

- Не променя външното състояние (променливи, структури, файлове, база данни).
- Не зависи от външно състояние.
- Не извършва входно-изходни операции (print, логване, четене от файл).
- За едни и същи входни аргументи винаги връща един и същ резултат.

Такава функция се нарича чиста функция.

Постъпково разяснение:

- Не променя външното състояние (променливи, структури, файлове, база данни).

Пример: Функция, която ЗАВИСИ от външно състояние (изчисляване на ДДС)

```
tax_rate = 0.20

def calculate_price(price) do
  price + price * tax_rate
end
```

Какъв е проблемът?

- ✓ tax_rate е външна променлива
 - ✓ Ако някой я промени, резултатът се променя
- Т.е. Функцията calculate_price(100) няма гарантиран резултат

Следователно: Функцията зависи от външно състояние tax_rate = 0.20

Същият пример, но БЕЗ външно състояние (чиста функция)

```
def calculate_price(price, tax_rate) do
  price + price * tax_rate
end
```

Защо това е правилно?

Всичко необходимо идва чрез аргументи

calculate_price(100, 0.20) винаги връща 120

Следователно: Няма външна зависимост

- Не зависи от външно състояние.
- Кога една функция зависи от външно състояние?
- Ако резултатът ѝ не зависи само от аргументите, а и от:
- ✓ глобални променливи (в примера по горе)
 - ✓ стойности, променени от други функции
 - ✓ текущо време
 - ✓ вход от потребителя
 - ✓ съдържание на файл или база данни

Следователно: Ако някое от горепосочените неща влияе на резултата тогава, функцията НЕ е чиста.

- Не извършва входно-изходни операции (print, логване, четене от файл).
- За едни и същи входни аргументи винаги връща един и същ резултат.

Пример БЕЗ странични ефекти (чиста функция)

```
def add(a, b) do
  a + b
end
```

Защо няма странични ефекти?

- Не променя нищо извън себе си
- Не чете външни данни
- Не отпечатва, не записва

add(2, 3) винаги връща 5

Пример СЪС странични ефекти

```
total = 0
def add_to_total(x) do
  total = total + x
end
```

Проблеми:

- Променя на външна променлива чрез функцията
- Резултатът зависи от предишни извиквания
- Трудно се тества и разбира

Друг пример за страничен ефект – вход/изход

```
def add_and_print(a, b) do
  result = a + b
  IO.puts(result)
end
```

Защо има страничен ефект в този фрагмент?

- Функцията извежда резултата на екрана, което е промяна на “околния свят”

4. Езици за функционално програмиране

Езиците за функционално програмиране прилагат принципите на функционалната парадигма. Някои от тях са чисто функционални и не допускат странични ефекти. Други са мултипарадигмени и комбинират функционални и императивни елементи. Функционалните езици използват неизменяеми структури от данни. Те поддържат мощни типови системи. Често предоставят автоматична оптимизация на рекурсивни извиквания. Подходящи са за конкурентни и разпределени системи. Използват се както в академични среди, така и в индустрията. Много съвременни езици заимстват идеи от функционалното програмиране. Функционалните езици играят важна роля в развитието на софтуерните технологии.

5. Програмна парадигма. Езици, парадигми и концепции

Програмната парадигма е фундаментален подход към програмирането, основан на математическа теория или съгласуван набор от принципи. Тя определя начина, по който се моделират проблемите, как се организира програмният код и как се интерпретират основните концепции.

Езиците за програмиране са конкретни реализации, които поддържат една или повече парадигми.

Концепциите (напр. функция, състояние, тип, обект) са най-малките градивни елементи.

- Една парадигма използва и организира концепциите по специфичен начин.
- Един език може да поддържа няколко парадигми (мултипарадигмен подход).
- Парадигмите са по-малко на брой от езиците.
- Изборът на парадигма влияе пряко върху архитектурата и дизайна на софтуера.



5.1. Класификации на програмните парадигми

5.1.1. Класификация I – основни парадигми

- Императивна – програма като последователност от команди
- Функционална – програма като оценка на функции
- Логическа – програма като система от факти и правила
- Обектно-ориентирана – програма като взаимодействие между обекти и класове

Функционалната и логическата парадигма се разглеждат като декларативни.

5.1.1.1. Императивна парадигма (ИП)

- Фокусира се върху това как се постига резултатът.
- Използва променливи и промяна на състоянието.
- Последователността на инструкциите е от решаващо значение.
- Тясно свързана е с фон-Ноймановата архитектура.
- Типични конструкции са присвояване, цикли и условни оператори.

5.1.1.2. Функционална парадигма (ФП)

- Основава се на математическата теория на функциите.
- Използва неизменяеми данни.
- Функциите са основна абстракция и са първокласни стойности.
- Акцентът е върху резултата, а не върху реда на изпълнение.
- Насърчава декларативен стил на програмиране.

5.1.1.3. Логическа парадигма (ЛП)

- Програмата описва факти и правила.
- Изчисленията се извършват чрез логически извод.
- Подходяща за системи, базирани на знания.
- ЛП е декларативна по същност си.

5.1.1.4. Обектно-ориентирана парадигма (ООП)

- Моделира реалния свят чрез обекти и класове.
- Основни принципи са капсулация, наследяване и полиморфизъм.
- Подходяща за големи и сложни софтуерни системи.

5.1.2. Класификация II

- Императивна
- Процедурна
- Обектно-ориентирана
- Декларативна

Процедурната парадигма е разширение на императивната.

Задача 1. Да се намери сумата от елементите в списък, като се използва Enum.reduce/3, който обхожда последователно списъка [1, 2, 3, 4], като започва с начална стойност 0 и към нея прибавя последователно всеки следващ елемент. Накрая получената сума (10) се записва в променливата sum и се отпечатва с IO.puts. Реализацията да е в ИМП, ФП, ЛП и ООП

1. Императивна парадигма (в стил Elixir) - постъпково логическа промяна на състояние

```
list = [1, 2, 3, 4]
sum = 0
sum =
Enum.reduce(list, sum, fn element, accumulator -> accumulator + element end)
IO.puts(sum)
```

Функцията Enum.reduce/3 приема три аргумента. Първият аргумент е списъкът list, който ще бъде обхождан, вторият е началната стойност sum (акумулатор), а третият е анонимна функция с два параметъра – текущия елемент (element) и натрупаната до момента стойност (accumulator).

Императивното програмиране използва натрупване на резултат. Въпреки, че Elixir не позволява истинска мутация, логиката имитира промяна на състояние. Ясно се виждат стъпките на алгоритъма.

2. Функционална парадигма - чиста функция, рекурсия, без състояние

```
defmodule FunctionalSum do
  def sum([]) do
    0
  end

  def sum([head | tail]) do
    head + sum(tail)
  end
end

result = FunctionalSum.sum([1, 2, 3, 4])
IO.puts(result)
```

Този код дефинира модул FunctionalSum с рекурсивна функция sum/1, която изчислява сумата на елементите в списък. Първата клауза обработва празен списък и връща 0, а втората разделя списъка на първи елемент и остатък и ги сумира рекурсивно. Накрая функцията се извиква със списъка [1, 2, 3, 4], а полученият резултат 10 се отпечатва на екрана.

В този вариант няма променливи, които се променят. Функцията винаги връща един и същ резултат за един и същ вход. Рекурсията замества циклите.

3. Логическа (декларативна) парадигма – модел в Elixir - описва се условие, което трябва да е вярно

```
defmodule LogicalSum do
  def sum_is_correct?(list, expected_result) do
    Enum.sum(list) == expected_result
  end
end

list = [1, 2, 3, 4]
expected = 10

result = LogicalSum.sum_is_correct?(list, expected)
IO.puts(result)
```

Този код реализира декларативен (логически) стил, като проверява дали дадено твърдение е вярно. Функцията sum_is_correct?/2 не описва алгоритъм, а проверява логическото условие дали сумата на елементите в списъка е равна на очаквания резултат. В края се извежда true, защото сумата на листа със стойности [1, 2, 3, 4] е равна на 10.

Не се описва алгоритъмът за сумиране. Описва се логическа зависимост: „Резултатът е верен, ако сумата на списъка е равна на очакваната стойност“. Това е декларативен, логически стил.

4. Обектно-ориентирана парадигма (ООП-подобен стил в Elixir) - данните и поведението са обединени

```
defmodule Calculator do
  defstruct numbers: []

  def sum(%Calculator{numbers: numbers}) do
    Enum.sum(numbers)
  end
end

calculator = %Calculator{numbers: [1, 2, 3, 4]}
result = Calculator.sum(calculator)

IO.puts(result)
```

Този код използва обектно-ориентиран подход, реализиран чрез структура и модул в Elixir. Данните (списъкът numbers) са капсулирани в структурата Calculator, а функцията sum/1 реализира поведението върху тези данни. Създава се обект от тип Calculator, изчислява се сумата на числата и резултатът 10 се извежда на екрана.

Данните (numbers) са капсулирани в структура. Функцията sum/1 работи върху този „обект“. Това наподобява клас и метод в ООП.

6. Императивни и дескриптивни програмни езици

6.1. Императивно програмиране (ИП)

- Императивното програмиране описва как да се постигне даден резултат.
- Програмата е последователност от инструкции.
- Състоянието се променя по време на изпълнение.
- Широко използвано и лесно за усвояване.

Императивни езици: C, C++, Java, C#, Python и др.

6.2. Дескриптивно програмиране (ДП)

- Дескриптивното програмиране описва какво трябва да бъде получено.
- Не се дефинира конкретният алгоритъм.
- Използва знания, правила и абстрактни конструкции.
- Типично за функционални и логически езици.

6.3. Съпоставка ИП – ДП

ИП е с фокус върху „КАК“

ДП е с фокус върху „КАКВО“

Императивният код е по-дълъг и по-подробен.

Декларативният код е по-кратък и по-изразителен.

Предимства на ИП са - яснота, контрол, лесно обучение.

Недостатъци на ИП са - сложност при големи системи.

Предимства на ДП са - по-малко код, по-добра поддръжка и по-малко грешки.

7. Функционално програмиране – дефиниция

- Функционалното програмиране е парадигма, при която изчислението се разглежда като оценка на функции.
- Избягва промяна на състоянието и странични ефекти.
- Фокусира се върху резултатите, а не върху действията.
- Подходящо е за изграждане на мащабируеми и надеждни системи.

7.1. Теоретичен модел

- Основава се на λ -смятането.
- Използва чисти функции и рекурсия.

7.1.2. Предимства и недостатъци

Предимства:

- по-лесно тестване
- по-малко грешки
- добра паралелизация

Недостатъци:

- по-стръмна крива на обучение
- понякога по-ниска производителност

Извод: ФП е особено подходящо за сложни и конкурентни системи.

7.2. Видове функционални езици

- Чисто функционални
- Нечисто функционални (мултипарадигмени)

7.3. История и езици

LISP, ML, Hope, Haskell, Clojure, Elm, Elixir, JavaScript, Kotlin, F#, Idris, Scala. Много съвременни езици поддържат функционален стил частично.

8. Основни принципи на функционалното програмиране

Immutability – неизменяеми данни
 Disciplined state – контролирано състояние
 Pure functions – без странични ефекти
 First-class и higher-order functions (Pipe оператор)
 Type systems – строги и изразителни типове
 Referential transparency – еднакъв вход и еднакъв изход

ПРИМЕРИ:

1. Immutability – неизменяеми данни

```
list1 = [1, 2, 3]
list2 = [0 | list1]
```

list1 не се променя, а се създава нов списък list2.

2. Disciplined state – контролирано състояние

```
def increment(value) do
  value + 1
end
```

Състоянието не е глобално, а се предава и връща чрез аргументи.

3. Pure functions – без странични ефекти

```
def add(a, b) do
  a + b
end
```

Функцията не чете и не променя външно състояние и винаги връща един и същ резултат.

4. First-class и Higher-order functions (Pipe оператор)

```
[1, 2, 3]
> Enum.map(fn x -> x * 2 end)
> Enum.sum()
```

Функциите се подават като аргументи и се композират чрез pipe (|>).

5. Type systems – типова система (в Elixir)

```
defmodule Math do
  @spec add(integer(), integer()) :: integer()
  def add(a, b), do: a + b
end
```

@spec описва очакваните типове и подпомага анализа на кода.

6. Referential transparency – еднакъв вход - еднакъв изход

```
def square(x) do
  x * x
end
```

square(4) винаги връща 16, независимо кога и колко пъти се извиква.

Таблица 1. Съпоставка между ФЕ за програмиране

Език	Тип език	Чисто ФП	Типова система	Имутабилност (непроменяща се)	Странични ефекти	Основно предназначение на езика
LISP	Ф	не	Динамична	Частично	Допуска	Академичен, AI
ML	Ф	не	Статична, силна	Да	Ограничени	Академичен, системен
Норе	Ф	да	Статична	Да	Не	Образователен
Haskell	Ф	да	Статична, силна	Да	Не (контролирани чрез монади)	Академичен, индустриален
Clojure	МП	не	Динамична	Да	Контролирани	Разпределени системи

Elm	Ф	да	Статична	Да	Не	Уеб интерфейси
Elixir	МП	не	Динамична	Да	Контролирани	Разпределени системи
JavaScript	МП	не	Динамична	Не	Да	Уеб, фронтенд/ бекенд
Kotlin	МП	не	Статична	Частично	Да	JVM, мобилни приложения
F#	Ф	не	Статична, силна	Да	Ограничени	.NET платформа
Idris	Ф	да	Зависими типове	Да	Не	Академичен
Scala	МП	не	Статична	Частично	Да	JVM, големи системи

**Имутабилност е свойството на данните да не се променят след създаването им, като всяка „промяна“ води до създаване на нова стойност.*

9. Добри практики във функционалното програмиране

- Използване на чисти функции.
- Минимизиране на състоянието.
- Композиция на функции.
- Ясен и декларативен код.

10. Заключение

- Програмните парадигми определят начина на мислене и моделиране на проблемите.
- Императивният и декларативният подход имат различни силни страни.
- Функционалното програмиране предоставя мощни средства за изграждане на надежден и мащабируем софтуер.
- Изборът на подходяща парадигма е ключов за качеството на софтуерната система.